UITS

Future will be better than thy past

An initiative of PHP Family

University of Information Technology & Sciences

Compiler Lab

Lab Report 3

Course Title : Compiler Lab

Course Code : CSE0613322

Submitted to:

Teacher's Name	Designation
Sonia Afroz	Lecturer
Tasneen Tanvir	Lecturer

Submitted by:

Student Name : Gaus Saraf Murady

Student ID : 0432410005101088

Semester : Autumn 2026

Batch : 55

Department : CSE

Date of Submission: 22.08.26

Experiment No. : 3

Experiment Name: C++ Program for recognizing types of comments and finding number of lines in comments

---

1. Manual Identifier and Operator Finding

```
#include <stdio.h>
int main() {
    int a = 10;
    int b = 20;
    int sum;
    sum = a + b;
    if (sum > 20) {
    printf("Sum = %d", sum);
    }
    return 0;
}
```

a. Identify all identifiers in the program.

Ans: Identifiers: “a”, “b”, “sum”

b. Identify all operators and classify them as arithmetic, assignment, relational, increment/decrement, or logical.

Ans: Assignment: “=”, Arithmetic: “+”, Relational: “>”

c. Identify the keywords and constants separately.

Ans: Keywords: “int”, “if”, “printf”, “return”, Constants: 10, 20, 0

2. Identifier Validation

Determine whether each identifier is valid or invalid according to C language rules. Provide a short explanation for every answer.

No.	Identifier	Valid/Invalid	Reason
1	total	valid	Not a keyword
2	student_name	valid	Multiple terms, separated by underscore (snake_case)
3	2value	invalid	Starts with a digit
4	_count	valid	Starts with an underscore
5	totalMarks	valid	Multiple terms separated by capitalization from the second term (camelCase)
6	student-name	valid	Multiple terms separated by hyphen (kebab-case)
7	float	invalid	Keyword
8	value2	valid	Ending with a digit
9	my variable	invalid	Separated by blank space
10	result_2026	valid	Multiple terms, separated by underscore (snake_case)

Code for Tasks 3 to 5:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
  cout << "\n\n[ Code does not recognize prefix increment or decrement. Also, "
          "strictly add spaces before endlines `;` ]\n\nEnter an expression: ";
  string text, inc_text;
  getline(cin, text);
  text += " ";

  vector<string> id, op, assig, arith, relat, logic, inc, dec, key, con;

  string temp = "";
  for (int i = 0; i < text.size(); i++) {
    if (text[i] == ' ') {
      if (!temp.empty()) {
        if (temp == "+" || temp == "-" || temp == "*" || temp == "/" ||
            temp == "=" || temp == "<" || temp == ">" || temp == "<=" ||
            temp == ">=" || temp == "!=" || temp == "==" || temp == "||" ||
            temp == "&&" || temp == "~" || temp == "!" ||
            temp[temp.size() - 1] == '+' || temp[temp.size() - 1] == '-' ||
            temp == "int" || temp == "float" || temp == "double" ||
            temp == "char" || temp == "bool" || temp == "void" ||
            temp == "auto" || temp == "nullptr" || temp == "if" ||
            temp == "else" || temp == "for" || temp == "while" ||
            temp == "do" || temp == "switch" || temp == "case" ||
            temp == "default" || temp == "break" || temp == "continue" ||
            temp == "return" || temp == "class" || temp == "struct" ||
            temp == "public" || temp == "private" || temp == "protected" ||
            temp == "this" || temp == "virtual" || temp == "friend" ||
            temp == "new" || temp == "delete" || temp == "const" ||
            temp == "static" || temp == "namespace" || temp == "using" ||
            all_of(temp.begin(), temp.end(), ::isdigit)) {
          // op.push_back(temp);
          if (temp == "=") {
            assig.push_back(temp);
            op.push_back(temp);
          } else if (temp == "+" || temp == "-" || temp == "*" || temp == "/") {
            arith.push_back(temp);
            op.push_back(temp);
          } else if (temp == "<" || temp == ">" || temp == "<=" ||
                     temp == ">=" || temp == "!=" || temp == "==") {
            relat.push_back(temp);
```

```
    op.push_back(temp);
  } else if (temp == "||" || temp == "&&" || temp == "~" ||
             temp == "!") {
    logic.push_back(temp);
    op.push_back(temp);
  } else if (temp.size() >= 2 && ((temp[temp.size() - 2] == '+' &&
                                   temp[temp.size() - 1] == '+') ||
                                  (temp[temp.size() - 2] == '-' &&
                                   temp[temp.size() - 1] == '-'))) {
    // inc_text += temp;
    for (int j = 0; j < temp.size() - 1; j++) {

      if (temp[j] == '+') {
        inc_text += "++";
        inc.push_back(inc_text);
        break;
      } else if (temp[j] == '-') {
        inc_text += "--";
        dec.push_back(inc_text);
        break;
      }
    }
    inc_text = "";
  } else if (temp == "int" || temp == "float" || temp == "double" ||
             temp == "char" || temp == "bool" || temp == "void" ||
             temp == "auto" || temp == "nullptr" || temp == "if" ||
             temp == "else" || temp == "for" || temp == "while" ||
             temp == "do" || temp == "switch" || temp == "case" ||
             temp == "default" || temp == "break" ||
             temp == "continue" || temp == "return" ||
             temp == "class" || temp == "struct" || temp == "public" ||
             temp == "private" || temp == "protected" ||
             temp == "this" || temp == "virtual" || temp == "friend" ||
             temp == "new" || temp == "delete" || temp == "const" ||
             temp == "static" || temp == "namespace" ||
             temp == "using") {
    key.push_back(temp);
  } else if (all_of(temp.begin(), temp.end(), ::isdigit)) {
    con.push_back(temp);
  }
} else if (temp != "(" && temp != ")" && temp != ";" && temp != "{" &&
           temp != "}") {
  if (find(id.begin(), id.end(), temp) == id.end()) {
    id.push_back(temp);
  }
}
temp = "";
```

```
    }
  } else {
    temp.push_back(text[i]);
  }
}

cout << endl << "Identifiers (" << id.size() << "):\n";
for (int i = 0; i < id.size(); i++) {
  cout << id[i] << endl;
}

cout << endl << "Operators (" << op.size() << "):\n";
for (int i = 0; i < op.size(); i++) {
  cout << op[i] << endl;
}

cout << endl << "Operator Classification:\n";

cout << "Assignment (" << assig.size() << "):\n";
for (int i = 0; i < assig.size(); i++) {
  cout << "  " << assig[i] << endl;
}

cout << "Arithmetic (" << arith.size() << "):\n";
for (int i = 0; i < arith.size(); i++) {
  cout << "  " << arith[i] << endl;
}

cout << "Relational (" << relat.size() << "):\n";
for (int i = 0; i < relat.size(); i++) {
  cout << "  " << relat[i] << endl;
}

cout << "Logical (" << logic.size() << "):\n";
for (int i = 0; i < logic.size(); i++) {
  cout << "  " << logic[i] << endl;
}

cout << "Increment (" << inc.size() << "):\n";
for (int i = 0; i < inc.size(); i++) {
  cout << "  " << inc[i] << endl;
}

cout << "Decrement (" << dec.size() << "):\n";
for (int i = 0; i < dec.size(); i++) {
  cout << "  " << dec[i] << endl;
}
```

```
  cout << "Keyword (" << key.size() << "):\n";
  for (int i = 0; i < key.size(); i++) {
    cout << "  " << key[i] << endl;
  }

  cout << "Constant (" << con.size() << "):\n";
  for (int i = 0; i < con.size(); i++) {
    cout << "  " << con[i] << endl;
  }

  return 0;
}
```

3. Write a Program that accepts a line of source code as input and finds: (1) all identifiers, (2) arithmetic operators, (3) relational operators, (4) logical operators, and (5) assignment operators.

Input :

```
total = a + b * 10 ;
```

Output :

```
[ Code does not recognize prefix increment or decrement. Also, strictly add spaces before endlines `;` ]

Enter an expression: total = a + b * 10 ;

Identifiers (3):
total
a
b

Operators (3):
=
+
*

Operator Classification:
Assignment (1):
  =
Arithmetic (2):
  +
  *
Relational (0):
Logical (0):
Increment (0):
Decrement (0):
Keyword (0):
Constant (1):
  10
PS C:\Users\gsmur\Documents\GitHub\UITS\Compiler-Lab-Codes\Assignments>
```

4. Advanced Test Case to process the following program fragment and verify that your program correctly identifies every identifier and operator.

Input :

```
int result = ( a + b ) * c ; if ( result >= 100 && result != 150 ) { result++ ; }
```

Output :

```
[ Code does not recognize prefix increment or decrement. Also, strictly add spaces before endlines `;` ]

Enter an expression: int result = ( a + b ) * c ; if ( result >= 100 && result != 150 ) { result++ ; }

Identifiers (4):
result
a
b
c

Operators (6):
=
+
*
>=
&&
!=

Operator Classification:
Assignment (1):
  =
Arithmetic (2):
  +
  *
Relational (2):
  >=
  !=
Logical (1):
  &&
Increment (1):
  ++
Decrement (0):
Keyword (2):
  int
  if
Constant (2):
  100
  150
```

5. Token Classification to classify each token below as Keyword, Identifier, Constant, Arithmetic Operator, Assignment Operator, Relational Operator, Logical Operator, or Increment/Decrement Operator.

Input :

```
int student = 25 + marks > 20 && total ++
```

Output :

```
[ Code does not recognize prefix increment or decrement. Also, strictly add spaces before endlines `;` ]

Enter an expression: int student = 25 + marks > 20 && total ++

Identifiers (3):
student
marks
total

Operators (4):
=
+
>
&&

Operator Classification:
Assignment (1):
  =
Arithmetic (1):
  +
Relational (1):
  >
Logical (1):
  &&
Increment (1):
  ++
Decrement (0):
Keyword (1):
  int
Constant (2):
  25
  20
```

Discussion:

Through this assignment, I learned how to work with empty spaces, use strings as arrays of strings (or as vectors as improvement on it), how to identify increment and decrement syntaxes and more. Other learnings are summarized below. (Note that, some parts of it required AI assistance, which are documented in the GitHub repository for this lab: [Compiler-Lab-Codes/Assignments/learning at main · b1tranger/Compiler-Lab-Codes]( ))

- Increment & Decrement Separation: Divided unary operators into dedicated inc (++) and dec (--) categories with distinct storage vectors and classification counts.
- Punctuation & Delimiter Filtering: Applied compound Boolean filtering (&&) to isolate non-identifier syntax symbols such as (,), {, }, and ;.
- Numeric Literal Identification: Integrated std::all_of with ::isdigit to classify integer constants separately from user identifiers.
- Identifier Deduplication: Utilized std::find() containment checks prior to insertion to maintain unique identifier sets while preserving chronological appearance order.
- Keyword Classification: Established a reserved-word lookup table to prevent standard C++ control structures (if, else, while, int, etc.) from being misclassified as user identifiers.

Conclusion:

I was able to complete the assignment successfully. While I did the assignment in C++, I am aware that C and Python have their own way of doing the same thing. And I will have to review them for my own growth.